ROOMFID · DESPLIEGUE

Cambiar la IP de la Raspberry en pruebas

Qué direcciones se tocan al montar el sistema fuera del laboratorio, y con qué script.

01 Lo primero: qué IP se cambia

En el proyecto conviven tres grupos de direcciones. Confundirlos es el error más frecuente, así que conviene fijarlo antes de abrir ningún fichero.

DIRECCIONES	DÓNDE VIVEN	¿SE CAMBIAN?
`192.168.1.155` `.160` … `.167`	Red interna de Docker (`docker-compose.yml`)	**Nunca.** Son internas: nadie de fuera las ve. Están fijas a propósito para que las configuraciones heredadas del sistema original funcionen sin reescribirse.
La de la propia Pi	Configuración de red de la Raspberry	**Sí.** Es la que cambia al mover el sistema de sitio.
`.157` `.158` `.159` `.205`	`topologia.json`	**Solo si el lector cambió de dirección físicamente.**

En una frase

No toques las IP del `docker-compose.yml` . Al montar en pruebas solo se cambian **la IP de la Raspberry** y, si han cambiado de sitio, **las de los lectores en** `topologia.json` .

02 Elige el escenario

Hay dos formas de montar, y la IP de la Pi se gestiona de forma distinta en cada una. Decide primero: no son intercambiables.

A Banco de pruebas

CARPETA

`rep_server/banco/`

RED

Aislada: switch propio, solo la Pi y los lectores

B Prueba en campo

CARPETA

`rep_server/campo/`

RED

La real del almacén, con el sistema en marcha

IP DE LA PI

Toma la `.155`, la del servidor original

LECTORES

Se recablean al switch del banco

PARA QUÉ

Probar el sistema nuevo con las antenas reales, de punta a punta

IP DE LA PI

Una IP **libre** de la `192.168.1.x`

LECTORES

Se quedan donde están

PARA QUÉ

Observar en paralelo sin interferir (modo sombra)

03 Escenario A · la Pi toma la `.155`

Aquí la Raspberry **suplanta al servidor original**: toma su misma IP para que los lectores, que la tienen configurada de fábrica, publiquen contra ella sin reconfigurarse.

Antes de nada

No pueden existir dos equipos con la `192.168.1.155` en la misma red. Si la Pi toma esa IP conectada a la red real, **se rompe el sistema de producción**. Por eso la prueba va en un switch aparte.

`ip.sh` comprueba si esa IP ya responde y se niega a seguir, pero no te fíes solo de eso: verifica el cableado con la vista.

Paso 1 · Sacar el volcado (antes de cambiar la IP)

El volcado necesita llegar al servidor original, así que se saca **con la Pi todavía en la red normal**. Después ya no habrá forma.

```
# En la Pi, aún con su IP de siempre
cd ~/roomfid/rep_server/banco
cp .env.ejemplo .env          # revisar las claves
./scripts/volcado.sh          # deja volcado/produccion.sql
```

La dirección del servidor de origen se declara en `.env`:

```
ORIGEN_PG_HOST=192.168.1.155   # solo para el volcado
ORIGEN_PG_PORT=5432
```

Si no hay acceso, sirve cualquier volcado de `gestion_rfid` copiado a mano en `banco/volcado/produccion.sql`.

Paso 2 · Recablear

Mueve los cuatro lectores del switch del almacén al del banco, y conecta ahí la Pi.

Comprobar antes de desplazarse

Comprobar antes de desplazarse

Si los lectores se alimentan por PoE, **el switch del banco tiene que ser PoE**. Sin PoE no arrancan y el viaje se pierde.

Paso 3 · Dar la IP

```
cd ~/roomfid/rep_server/banco

./scripts/ip.sh ver       # qué IP tiene ahora
./scripts/ip.sh on        # añade la .155 hasta el próximo reinicio
./scripts/ip.sh fijar     # o permanente: sobrevive a reinicios
```

`ip.sh on`

Añade la `.155` **además** de la que ya tenga. Se pierde al reiniciar. Es la opción para una prueba de unas horas.

`ip.sh fijar`

La fija en NetworkManager. Sobrevive a reinicios y cortes de luz; con los contenedores, que también vuelven solos, el banco se recupera sin que nadie toque nada. Es la opción si el montaje se queda varios días.

`ip.sh off`

La quita, tanto la temporal como la permanente.

`ip.sh ver`

Muestra las IP actuales y las permanentes. No cambia nada.

Los tres primeros aceptan la interfaz como segundo argumento, si no es `eth0`:

```
./scripts/ip.sh on eth1
```

La Pi conserva su IP original además de la `.155`, así que puedes seguir entrando por SSH por la dirección de siempre.

Paso 4 · Arrancar

```
./scripts/arrancar.sh     # la primera vez restaura el volcado
./scripts/estado.sh       # salud, ciclos y movimientos
```

Sin internet en la nave

`arrancar.sh` no construye ni descarga nada. Las imágenes tienen que estar ya construidas: ejecuta `./scripts/construir.sh` en casa, con conexión (15–30 min la primera vez). Si falta alguna, `arrancar.sh` avisa y no arranca.

Efecto en el sistema original

Mientras los lectores están en el banco, el almacén se queda sin ellos: el batch original marcará todo como no encontrado (`Stock=0`) a los cinco minutos y lo asignará al último carnet que pasara por la puerta.

Al devolver los lectores, las lecturas vuelven a poner el material en `Stock=1` , pero **`EPC_User` se queda con el valor de la prueba**. Hay que avisar al cliente, o pausar Node-RED en la Raspberry original si lo autoriza.

04 Escenario B · la Pi con una IP libre

Aquí la Pi se conecta a la red del almacén **sin suplantar a nadie**: escucha el mismo broker que el sistema en marcha y calcula en paralelo. No enciende lectores ni escribe en la base de producción.

Paso 1 · Fijar la IP en la Raspberry

Pide al cliente una IP libre de la `192.168.1.x` , fija o reservada en el DHCP. No uses la `.155` ni las de los lectores (`.157` , `.158` , `.159` , `.205`).

Aquí **no se usa `ip.sh`** — ese script es exclusivo del banco. Se configura con las herramientas de la propia Pi:

```
# Ver las conexiones y cómo están configuradas
nmcli con show

# Fijar IP estática (ajusta el nombre de la conexión y la dirección)
sudo nmcli con modify "Wired connection 1" \
     ipv4.method manual \
     ipv4.addresses 192.168.1.201/24 \
     ipv4.gateway 192.168.1.1 \
     ipv4.dns 192.168.1.1
sudo nmcli con up "Wired connection 1"

ip -4 -br addr show eth0        # comprobar
```

Paso 2 · Decirle dónde está el servidor real

Una sola variable en `campo/.env` :

```
cd ~/roomfid/rep_server/campo
cp .env.ejemplo .env
nano .env


SERVIDOR=192.168.1.155      # Mosquitto, PostgreSQL y API del sistema real
```

```
SERVIDOR=192.168.1.135      # mosquitto, PostgreSQL y API del sistema real
MQTT_PORT=1883
MQTT_USER=gestion_rfid
MQTT_PASS=                  # rellenar
API_KEY=                    # rellenar

PROD_PG_PORT=5432           # base de producción: SOLO lectura
PROD_PG_USER=rfidroom
PROD_PG_PASSWORD=           # rellenar

PG_PASSWORD=                # base PROPIA de la Pi: cualquier clave
```

Un solo sitio

`SERVIDOR` es la única variable donde se escribe la dirección del servidor real: de ahí la toman el broker y las conexiones de lectura. Si el servidor cambia de IP, cámbiala aquí y reinicia con `./scripts/arrancar.sh`.

Paso 3 · Puertos, si alguno choca

VARIABLE	POR OMISIÓN	SERVICIO
`PUERTO_PG`	`15432`	PostgreSQL propio de la Pi
`PUERTO_HEALTH`	`8090`	Salud de irfid
`PUERTO_VISOR`	`8020`	Visor web
`PUERTO_MONITOR`	`8080`	Monitor

Paso 4 · Arrancar y comparar

```
./scripts/volcado.sh      # copia la base de producción (lectura)
./scripts/arrancar.sh
./scripts/estado.sh
./scripts/comparar.sh     # contrasta el cálculo nuevo con el de producción
```

05 Si un lector cambia de IP

Se declara en el `topologia.json` de la carpeta que uses:

```
"lectores": [
  {
    "devmac": "3c:e4:b0:dc:ca:d7",    <-- identifica al lector DE VERDAD
    "devid": "Almacen-1",
    "devip": "192.168.1.157",         <-- cambiar aquí
```

```
    "ubicacion": "Almacen-1",
    "rol": "inventario",
    "orden_ciclo": 1
  },
```

La MAC es lo que cuenta

El sistema identifica cada lector **por su MAC**, no por su IP; la IP solo se usa para encenderlo y apagarlo. Si llega una trama con una MAC que no está en `topologia.json`, **se descarta** y el lector sale como `lectores_desconocidos` en `http://127.0.0.1:8090/health`. Es el primer sitio donde mirar si un lector «no hace nada».

Después, reinicia para que se recargue: `./scripts/arrancar.sh`

06 Comprobaciones, en orden

Si una falla, arréglala antes de pasar a la siguiente.

- [ ] **La Pi tiene la IP que debe**
  `ip -4 -br addr show eth0`
- [ ] **Los contenedores están sanos**
  `./scripts/estado.sh` y `docker compose ps` — todos *healthy*
- [ ] **Los lectores responden**
  `./scripts/lector.sh 157 status`, luego `start` y `stop`
- [ ] **Publican y se les reconoce**
  `curl -s http://127.0.0.1:8090/health | python3 -m json.tool` — `lectores_desconocidos` debe estar vacío
- [ ] **Las lecturas llegan a la base**
  Contar filas de `EPC_readings` de los últimos 5 minutos
- [ ] **Las interfaces abren**
  Visor `:8020` · Monitor `:8080` · API `:3000` · Mailpit `:8025` (solo banco)

Desde el portátil, conectado al mismo switch con una IP libre (por ejemplo `192.168.1.200`), sustituyendo `<ip-pi>` por la de la Raspberry.

07 Volver atrás

```
# 1. Parar el sistema de pruebas
cd ~/roomfid/rep_server/banco      # o campo/
./scripts/parar.sh
```

```
# 2. Quitar la IP del servidor (SOLO en el banco)
./scripts/ip.sh off

# 3. Comprobar que ya no la tiene
./scripts/ip.sh ver
```

1. **Devolver los cables** de los lectores al switch del almacén (escenario A).
2. **Reanudar Node-RED** en la Raspberry original, si se pausó.
3. **Avisar al cliente**: los `EPC_User` de ese periodo son de la prueba, no retiradas reales.

No olvidar

El paso 2 es imprescindible. Si la Pi conserva la `.155` y vuelve a la red real, chocará con el servidor original. Con `ip.sh fijar` la IP es permanente, y `ip.sh off` es la única forma de quitarla.

Para empezar de cero con otro volcado, `parar.sh -v` borra además la base de la Pi.

08 Cuando algo no va

SÍNTOMA	CAUSA PROBABLE	QUÉ HACER
`ip.sh on` dice «ALTO: ya responde en esta red»	La Pi está en la red real, no en la del banco	Revisa el cableado. Es la protección funcionando: **no la fuerces**.
Los lectores no publican nada	Siguen apuntando al servidor original, o no tienen alimentación	Comprueba que la Pi tiene la `.155` y que el switch da PoE si hace falta.
Un lector sale en `lectores_desconocidos`	Su MAC no coincide con `topologia.json`	Copia la MAC que indique `/health` y reinicia.